

Reviewer Pack — Cyclic Fourier Spread of Term-Vertex Incidence in Monotone k...

Entry f4d1273a6af3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 11:23:15 UTC

Cyclic Fourier Spread of Term-Vertex Incidence in Monotone k-CLIQUE DNFs

Entry ID: f4d1273a6af3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 11:23:15 UTC

1. Conjecture

Field A (mathematical branch): Representation theory of cyclic subgroups $C_n \leq S_n$ via character-Fourier decomposition of term-vertex incidence (monotone-GCT flavor, Ikenmeyer-style combinatorial reduction of Mulmuley-Sohoni symmetries) **Field B** (complexity object): Monotone DNF formula size / structure for the k-CLIQUE indicator (Razborov 1985 monotone setting, edges of K_n as variables)

Statement:

For a monotone DNF $F = T_1 \vee \dots \vee T_m$ on the edges of K_n , build $M(F) \in \mathbb{Z}^{m \times n}$ with $M(F)[i, v] := |\{e \in T_i : v \in e\}|$, and let $\tilde{M}(F)[i, j] := \sum_v M(F)[i, v] \cdot \exp(2\pi i \cdot jv/n)$ be its column-wise C_n -DFT. Define the cyclic-Fourier spread $\mu(F) := \#\{j \in \{0, \dots, n-1\} : \exists i \text{ with } |\tilde{M}(F)[i, j]| > 0\}$. Conjecture: for every monotone DNF F that computes the k-CLIQUE indicator on K_n with $n \geq 2k$ and $k \geq 3$, $\mu(F) \geq \lceil n/2 \rceil$; equivalently, no monotone DNF computing k-CLIQUE can confine its term-vertex incidence to fewer than half of the cyclic frequencies.

Rationale (proposer's reasoning):

In the GCT-monotone program one seeks symmetry-based obstructions: any formula for a highly symmetric function like the k-CLIQUE indicator must, after restricting the S_n action to a tractable cyclic subgroup C_n , exhibit a wide spread across irreducible C_n -characters because the k-subset orbit-structure couples to all nontrivial frequencies. A poly-size monotone DNF that 'collapses' incidence into few C_n -characters would correspond to a forbidden orbit-closure containment for the k-CLIQUE polynomial. Because μ is a complex-character invariant tied to roots of unity (not a polynomial-ring identity), it does not algebrize, and because it depends on the formula F (not the truth table) it dodges natural proofs.

Taxonomy category: GCT (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): bfd057beab5fd1bf

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 7 (n,k) configs $\times$ 30 seeds = 210 monotone DNFs that provably compute k-CLIQUE (canonical $\cup$ implied padding), compute $\mu(F)$ via column-wise C_n-FFT of M(F) with $|\text{entry}| > 1e-9$. SUPPORT iff $\mu(F) \geq \lfloor n/2 \rfloor$ on $\geq 90\%$ of the 210 trials AND on the canonical DNF for every (n,k); FALSIFIED iff any single k-CLIQUE-computing F yields $\mu(F) < \lfloor n/2 \rfloor$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - monotone DNF k-clique Fourier cyclic group representation lower bound - Razborov monotone clique character Fourier decomposition symmetric group - term-vertex incidence matrix monotone circuit clique cyclic DFT spectrum

Top relevant hits considered: - [<http://arxiv.org/abs/math/0509044v2>] Decomposing symmetric powers of certain modular representations of cyclic groups - [<http://arxiv.org/abs/2005.06373v2>] Counting Schur Rings over Cyclic Groups of Semi-prime Order - [<http://arxiv.org/abs/1801.08709v2>] Adaptive Lower Bound for Testing Monotonicity on the Line - [<http://arxiv.org/abs/0902.2349v1>] Defect of characters of the symmetric group - [<http://arxiv.org/abs/hep-ph/0610012v1>] Tevatron-for-LHC Report of the QCD Working Group - [<http://arxiv.org/abs/0810.1176v1>] A group with three real irreducible characters: answering a question of Moreto and Navarro - [<http://arxiv.org/abs/2311.04204v3>] Sharp Thresholds Imply Circuit Lower Bounds: from random 2-SAT to Planted Clique - [<http://arxiv.org/abs/2501.09545v1>] Hardness of clique approximation for monotone circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_k_clique_computing(dnf, k):
        n = len(dnf[0])
        for term in dnf:
            if len(term) != k:
                return False
```

```

    return True

def generate_k_clique_dnf(n, k):
    edges = [(i, j) for i in range(n) for j in range(i + 1, n)]
    clique_edges = random.sample(edges, k * (k - 1) // 2)
    dnf = []
    for edge in clique_edges:
        term = [edge[0], edge[1]]
        for other_edge in edges:
            if other_edge != edge and set(other_edge).issubset(set(edge)):
                term.append(other_edge[0])
                term.append(other_edge[1])
        dnf.append(term)
    return dnf

def generate_non_k_clique_dnf(n, k):
    edges = [(i, j) for i in range(n) for j in range(i + 1, n)]
    non_clique_edges = [e for e in edges if e not in random.sample(edges, k * (k - 1) // 2)]
    dnf = []
    for edge in non_clique_edges:
        term = [edge[0], edge[1]]
        for other_edge in edges:
            if other_edge != edge and set(other_edge).issubset(set(edge)):
                term.append(other_edge[0])
                term.append(other_edge[1])
        dnf.append(term)
    return dnf

def matrix_multiplication(A, B):
    m = len(A)
    n = len(B[0])
    p = len(B)
    result = [[sum(A[i][k] * B[k][j] for k in range(p)) for j in range(n)] for i in range(m)]
    return result

def gaussian_elimination(matrix):
    m, n = len(matrix), len(matrix[0])
    augmented_matrix = [row + [0] for row in matrix]
    for i in range(m):
        max_row = i
        for j in range(i + 1, m):
            if abs(augmented_matrix[j][i]) > abs(augmented_matrix[max_row][i]):
                max_row = j
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]
        pivot = augmented_matrix[i][i]
        for j in range(n + 1):
            augmented_matrix[i][j] /= pivot
        for j in range(m):
            if j != i:
                factor = augmented_matrix[j][i]
                for k in range(n + 1):
                    augmented_matrix[j][k] -= factor * augmented_matrix[i][k]
    return [row[:-1] for row in augmented_matrix]

def dft(M, n):
    m = len(M)

```

```

    result = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            sum_val = 0
            for v in range(n):
                sum_val += M[i][v] * math.cos(2 * math.pi * j * v / n)
                sum_val -= M[i][v] * math.sin(2 * math.pi * j * v / n) * 1j
            result[i][j] = sum_val
    return result

def cyclic_fourier_spread(M, n):
    m = len(M)
    M_hat = dft(M, n)
    count = sum(abs(entry) > 1e-9 for row in M_hat for entry in row)
    return count

test_cases = [(6, 3), (8, 3), (10, 3), (12, 3), (12, 4), (16, 4), (20, 4)]
results = []

for n, k in test_cases:
    canonical_dnf = generate_k_clique_dnf(n, k)
    non_k_clique_dnf = generate_non_k_clique_dnf(n, k)

    mu_canonical = cyclic_fourier_spread(canonical_dnf, n)
    mu_non_k_clique = cyclic_fourier_spread(non_k_clique_dnf, n)

    results.append({
        "n": n,
        "k": k,
        "mu_canonical": mu_canonical,
        "mu_non_k_clique": mu_non_k_clique
    })

mean_mu_canonical = sum(result["mu_canonical"] for result in results) / len(results)
mean_mu_non_k_clique = sum(result["mu_non_k_clique"] for result in results) / len(results)
support_fraction = sum(1 for result in results if result["mu_canonical"] >= n // 2) / len(results)

return {
    "metric_name": "cyclic_fourier_spread",
    "metric_value": mean_mu_canonical,
    "instances_tested": len(results),
    "conjecture_holds": support_fraction >= 0.9,
    "counterexample": "" if support_fraction >= 0.9 else f"n={results[0]['n']}, k={results[0]['k']}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_mu_canonical = sum(result["mu_canonical"] for result in results) / len(results)
    mean_mu_non_k_clique = sum(result["mu_non_k_clique"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["mu_canonical"] >= n // 2) / len(results)

```

```

if support_fraction >= 0.9:
    print(f"RESULT: SUPPORTED mean={mean_mu_canonical} std=NA support_fraction={support_fraction}")
elif any(result["mu_canonical"] < n // 2 for result in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if result["mu_canonical"] < n // 2)
    print(f"RESULT: FALSIFIED counterexample=n={results[0]['n']}, k={results[0]['k']} first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_adf4554a.py", line 132, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_adf4554a.py", line 105, in run_trial
    mu_canonical = cyclic_fourier_spread(canonical_dnf, n)
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_adf4554a.py", line 94, in cyclic_fourier_spread
    M_hat = dft(M, n)
             ~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_adf4554a.py", line 87, in dft
    sum_val += M[i][v] * math.cos(2 * math.pi * j * v / n)
                ~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an IndexError in the DFT routine before producing any $\mu(F)$ measurements, so neither the support nor the falsification criterion could be evaluated. | next: Fix the indexing bug in cyclic_fourier_spread/dft (ensure M is built as $m \times n$ with vertices indexed $0..n-1$) and rerun the 7×30 trial battery.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	225023
2	preregistration	claude_max	opus	0	0	14194
3	novelty	claude_max	opus	0	0	3925
4	novelty	claude_max	opus	0	0	10137
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18170
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19069
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13986
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17560
9	judge	claude_max	opus	0	0	4961

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 327026 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/f4d1273a6af3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f4d1273a6af3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f4d1273a6af3.tar.gz` (if generated)